Governor/Owner Agent Prototype — Weekend Requirements

Oct 29, 2026 · @Christian Matsoukis

Goal

Test whether a governing agent that never sees source code can land real changes through per-unit owner agents, using interface contracts and a test/type oracle as its only feedback.

The thesis: splitting intent from implementation fixes the context-bloat problem in single-agent coding. The governor holds system-level intent; each owner holds deep knowledge of one bounded unit. The prototype exists to find out whether contracts plus tests are a rich enough language for the governor to coordinate through, and where that language breaks.

The weekend ends with a transcript of governor-to-owner instructions and a yes/no on whether a cross-cutting rename landed without the governor reading a line of code.

Scope

One tiny TypeScript repo, one governor, one owner per module, `tsc` and vitest as the oracle, a loop capped at three rounds. Node for the harness, since that is the fastest path to a running prototype.

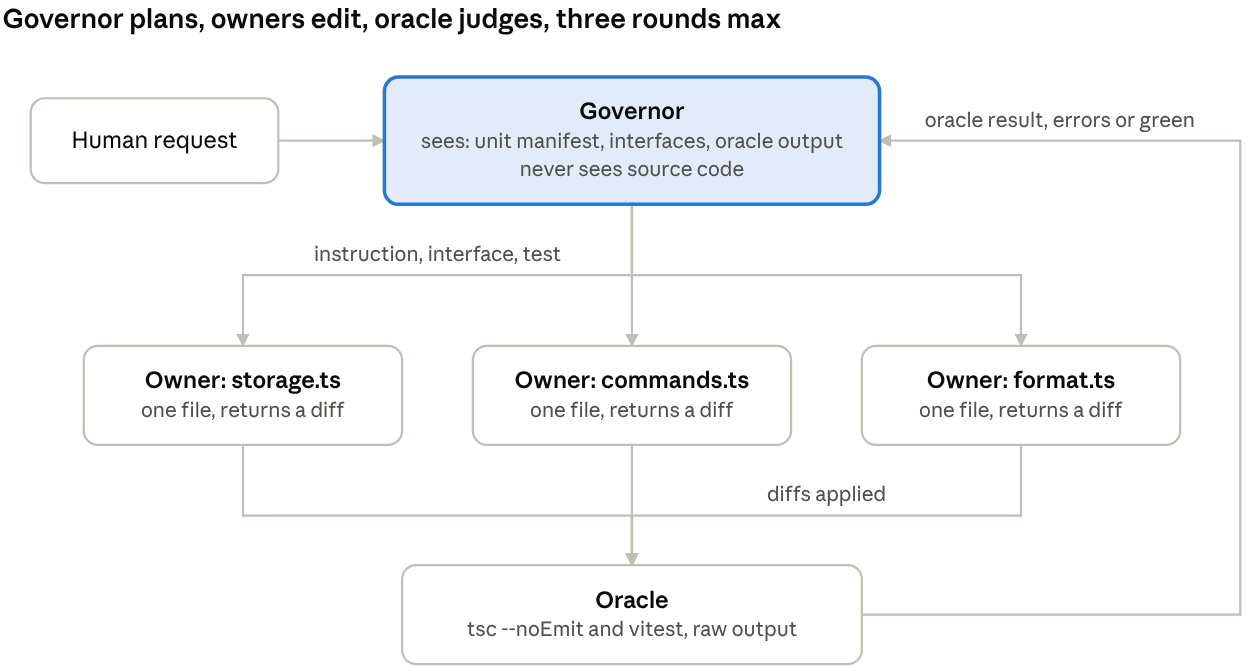
In scope

- Seed repo: a CLI todo app with 3–4 modules that have real cross-module dependencies (`storage.ts`, `commands.ts`, `format.ts`, plus `types.ts` for shared interfaces)
- Governor agent that plans in terms of units, interface changes, instructions and acceptance tests
- Owner agent that edits exactly one file and returns a diff
- Oracle that runs `tsc --noEmit` and the test suite and reports raw errors
- Orchestration loop with a hard round cap
- Full logging of every governor and owner message

Out of scope

- Agent birth and death (splitting or merging units)
- A middle coordination layer between governor and owners
- Persistent memory across runs
- Multi-repo, multi-language, or anything with real business logic
- Any UI beyond the terminal

Architecture



governor, owners, oracle · one loop, three rounds max

The governor turns the human request into a plan, each owner edits its own file in parallel, the oracle compiles and tests, and the raw result goes back to the governor, which accepts or reissues targeted instructions. The governor's only view of the system is the unit manifest and the oracle's output.

Governor context: the user's request, a manifest of units (name, one-line purpose, exported interface signatures), the last oracle result. Emits a plan: a list of `{unit, interfaceChange?, instruction, acceptanceTest?}`.

Owner context: its own file, the declared interfaces of the units it imports, the instruction. One tool: read its own file. Returns a diff.

Oracle: `tsc --noEmit` then the test suite. Returns raw errors, unfiltered. This is the governor's only verification channel.

Functional requirements

Seed repo

1. A TypeScript CLI todo app with `types.ts`, `storage.ts`, `commands.ts`, `format.ts`, each importing from at least one other.
2. A passing vitest suite with at least one test per module.
3. A `units.json` manifest: for each unit, its file path, one-line purpose, and exported interface signatures.

Governor

4. Receives the user request, the manifest, and the last oracle result. Never receives file contents.
5. Emits a plan as JSON: an array of `{unit, interfaceChange?, instruction, acceptanceTest?}`.
6. On a non-green oracle result, emits a revised plan targeting only the units implicated by the errors.
7. Emits `accept` when the oracle is green and its acceptance tests pass.

Owner

8. `runOwner(unit, instruction)` receives only its file, the interface signatures of units it imports, and the instruction.
9. Has one tool: read its own file.
10. Returns a unified diff against its file. Edits outside its file are rejected by the harness.
11. If the governor supplied an `interfaceChange`, the owner updates its exports to match and the manifest is regenerated.

Oracle

12. Applies all owner diffs, then runs `tsc --noEmit` and `vitest run`.
13. Returns raw stderr/stdout plus a pass/fail flag. No summarization.

Loop and logging

14. Owners run in parallel within a round.
15. Hard cap of 3 rounds per request; on exhaustion the harness stops and reports.

16. Every governor message, owner message, diff, and oracle result is appended to a per-run log file.

Saturday: the mechanics

Goal for the day: the loop runs end to end on a trivial request ("add a `--verbose` flag to `list`") and lands green.

- [ ] Scaffold the seed repo, tests passing, manifest generated
- [ ] Write the owner agent: single-file context, read tool, diff out
- [ ] Write the governor agent: manifest in, JSON plan out
- [ ] Write the oracle: apply diffs, run `tsc` and vitest, capture raw output
- [ ] Wire the loop with the 3-round cap and per-run logging
- [ ] Run the trivial request; fix the harness until it lands green

Keep every piece a plain function in one Node file until it works. Structure can come after the thesis survives.

Sunday: the experiments

Three requests, in order, each run from a clean checkout. Read the governor transcript after every run.

1. **Cross-cutting rename.** "Rename `Todo.done` to `Todo.completed` everywhere." The question: does interface-first coordination land this in one or two rounds, or does it collapse into round-trips?
2. **Behavior change, no interface change.** "Items with due dates sort before items without." The question: does the governor route this to one owner, or spray instructions across all of them?
3. **Ambiguous request.** "Make the output easier to read." The question: what does the governor do without a contract to anchor on? Does it invent an acceptance test, ask, or guess?

If time remains: give the governor a wrong manifest (stale signature) and watch whether the oracle feedback is enough for it to recover.

Success criteria

The thesis survives the weekend if the cross-cutting rename lands green in 2 rounds or fewer, the behavior change is routed to exactly one owner, and the governor never received a line of source.

Criterion	Pass	Fail
Rename lands green	⩽ 2 rounds	3 rounds or cap hit
Behavior change routing	1 owner instructed	2+ owners instructed
Governor isolation	No file contents in any governor message	Any leak
Ambiguous request	Governor asks or invents a testable contract	Governor guesses and ships

Record for each run: the request, round count, final oracle status, and the full governor-to-owner transcript. The transcript is the real output. Read it for what the intermediate language naturally wants to be: how much is interface, how much is test, how much is prose instruction, and where the governor had to say something it could not express in those terms.